Load GitHub Data to Supabase Using `dlt` in Python

TIP

We will be using the dlt PostgreSQL destination to connect to Supabase. You can get the connection string for your Supabase database as described in the Supabase Docs.

NEED HELP DEPLOYING THESE PIPELINES, OR FIGURING OUT HOW TO RUN THEM IN YOUR DATA STACK?

Join our Slack community or book a call with our support engineer Violetta.

This documentation explains how to load data from `GitHub` into `Supabase` using the open-source Python library `dlt`. The `dlt` library allows you to extract data on issues, pull requests, or events from any `GitHub` repository and load it into a destination of your choice, such as `Supabase`. `Supabase` is an open-source alternative to Firebase, offering a Postgres database, authentication, instant APIs, edge functions, real-time subscriptions, storage, and vector embeddings. For more detailed information about the `GitHub` API, visit GitHub's documentation.

`dlt` Key Features

- **Automated maintenance**: With schema inference and evolution and alerts, and with short declarative code, maintenance becomes simple. Learn more
- **Run it where Python runs**: On Airflow, serverless functions, notebooks. No external APIs, backends or containers, scales on micro and large infra alike. Learn more
- **User-friendly interface**: A declarative interface that removes knowledge obstacles for beginners while empowering senior professionals. Learn more
- **Detailed tutorial available**: A step-by-step guide to building a pipeline that loads data from the GitHub API into DuckDB. Follow the tutorial

Get GPT-4 Help

- **Community support**: Join the `dlt` community on Slack for help and support, and give the library a ⭐ on GitHub. Join Slack

Getting started with your pipeline locally

0. Prerequisites

`dlt` requires Python 3.8 or higher. Additionally, you need to have the `pip` package manager installed, and we recommend using a virtual environment to manage your dependencies. You can learn more about preparing your computer for dlt in our installation reference.

1. Install dlt

First you need to install the `dlt` library with the correct extras for `Supabase`:

```
pip install "dlt[postgres]"
```

The `dlt` cli has a useful command to get you started with any combination of source and destination. For this example, we want to load data from `GitHub` to `Supabase`. You can run the following commands to create a starting point for loading data from `GitHub` to `Supabase`:

```
# create a new directory
mkdir github_pipeline
cd github_pipeline
# initialize a new pipeline with your source and destination
dlt init github postgres
# install the required dependencies
pip install -r requirements.txt
```

The last command will install the required dependencies for your pipeline. The dependencies are listed in the `requirements.txt`:

```
dlt[postgres]>=0.3.25
```

You now have the following folder structure in your project:

```
github_pipeline/
├── .dlt/
│   ├── config.toml          # configs for your pipeline
│   └── secrets.toml         # secrets for your pipeline
├── github/                   # folder with source specific files
│   └── ...
├── github_pipeline.py        # your main pipeline script
├── requirements.txt         # dependencies for your pipeline
└── .gitignore               # ignore files for git (not required)
```

2. Configuring your source and destination credentials

The `dlt` cli will have created a `.dlt` directory in your project folder. This directory contains a `config.toml` file and a `secrets.toml` file that you can use to configure your pipeline. The automatically created version of these files look like this:

generated config.toml

```
# put your configuration values here

[runtime]
log_level="WARNING"  # the system log level of dlt
# use the dlthub_telemetry setting to enable/disable anonymous usage data
reporting, see https://dlthub.com/docs/telemetry
dlthub_telemetry = true
```

generated secrets.toml

```
# put your secret values and credentials here. do not share this file and do
not push it to github

[sources.github]
access_token = "access_token" # please set me up!

[destination.postgres]
dataset_name = "dataset_name" # please set me up!

[destination.postgres.credentials]
database = "database" # please set me up!
password = "password" # please set me up!
username = "username" # please set me up!
```

```
host = "host" # please set me up!
port = 5432
connect_timeout = 15
```

2.1. Adjust the generated code to your usecase

> **FURTHER HELP SETTING UP YOUR SOURCE AND DESTINATIONS**
>
> - Read more about setting up the `GitHub` source in our docs.
> - Read more about setting up the `Supabase` destination in our docs.

3. Running your pipeline for the first time

The `dlt` cli has also created a main pipeline script for you at `github_pipeline.py`, as well as a folder `github` that contains additional python files for your source. These files are your local copies which you can modify to fit your needs. In some cases you may find that you only need to do small changes to your pipelines or add some configurations, in other cases these files can serve as a working starting point for your code, but will need to be adjusted to do what you need them to do.

The main pipeline script will look something like this:

```
import dlt

from github import github_reactions, github_repo_events


def load_duckdb_repo_reactions_issues_only() -> None:
    """Loads issues, their comments and reactions for duckdb"""
    pipeline = dlt.pipeline(
        "github_reactions",
        destination='postgres',
        dataset_name="duckdb_issues",
        full_refresh=True,
    )
    # get only 100 items (for issues and pull request)
    data = github_reactions(
        "duckdb", "duckdb", items_per_page=100, max_items=100
    ).with_resources("issues")
```

```python
    print(pipeline.run(data))


def load_airflow_events() -> None:
    """Loads airflow events. Shows incremental loading. Forces anonymous
access token"""
    pipeline = dlt.pipeline(
        "github_events", destination='postgres', dataset_name="airflow_events"
    )
    data = github_repo_events("apache", "airflow", access_token="")
    print(pipeline.run(data))
    # if you uncomment this, it does not load the same events again
    # data = github_repo_events("apache", "airflow", access_token="")
    # print(pipeline.run(data))


def load_dlthub_dlt_all_data() -> None:
    """Loads all issues, pull requests and comments for dlthub dlt repo"""
    pipeline = dlt.pipeline(
        "github_reactions",
        destination='postgres',
        dataset_name="dlthub_reactions",
        full_refresh=True,
    )
    data = github_reactions("dlt-hub", "dlt")
    print(pipeline.run(data))


if __name__ == "__main__":
    load_duckdb_repo_reactions_issues_only()
    load_airflow_events()
    load_dlthub_dlt_all_data()
```

Provided you have set up your credentials, you can run your pipeline like a regular python script with the following command:

```
python github_pipeline.py
```

4. Inspecting your load result

You can now inspect the state of your pipeline with the `dlt` cli:

```
dlt pipeline github_events info
```

You can also use streamlit to inspect the contents of your `Supabase` destination for this:

```
# install streamlit
pip install streamlit
# run the streamlit app for your pipeline with the dlt cli:
dlt pipeline github_events show
```

5. Next steps to get your pipeline running in production

One of the beauties of `dlt` is, that we are just a plain Python library, so you can run your pipeline in any environment that supports Python >= 3.8. We have a couple of helpers and guides in our docs to get you there:

The Deploy section will show you how to deploy your pipeline to

- **Deploy with Github Actions**: Learn how to deploy a pipeline using Github Actions.
- **Deploy with Airflow**: Follow the steps to deploy a pipeline with Airflow and Google Composer.
- **Deploy with Google Cloud Functions**: Discover how to deploy your pipeline using Google Cloud Functions.
- **Explore other deployment options**: Check out other methods to deploy your pipeline here.

The running in production section will teach you about:

- **How to Monitor your pipeline**: Learn how to effectively monitor your `dlt` pipeline to ensure smooth operations and quickly identify any issues. Read more
- **Set up alerts**: Configure alerts to get notified about important events and potential issues in your `dlt` pipeline. Read more
- **Set up tracing**: Implement tracing to gain detailed insights into the execution of your `dlt` pipeline, helping you to diagnose and resolve issues efficiently. Read more

Available Sources and Resources

For this verified source the following sources and resources are available

Source `github_repo_events`

"GitHub repo events source provides data about activities and interactions within a repository."

Resource Name	Write Disposition	Description
`repo_events`	`append`	Retrieves all the repository events associated with the GitHub repository. This includes information about the actor (user who triggered the event), organization, payload (specific details about the event), and the repository itself.

Additional pipeline guides

- Load data from Klarna to ClickHouse in python with dlt
- Load data from Braze to Databricks in python with dlt
- Load data from MySQL to MotherDuck in python with dlt
- Load data from Bitbucket to Redshift in python with dlt
- Load data from Looker to ClickHouse in python with dlt
- Load data from Chess.com to Neon Serverless Postgres in python with dlt
- Load data from Attio to Supabase in python with dlt
- Load data from Adobe Commerce (Magento) to AWS Athena in python with dlt
- Load data from IBM Db2 to Databricks in python with dlt
- Load data from Pinterest to Snowflake in python with dlt

Edit this page

Last updated on **Aug 7, 2024** by **Zaeem Athar**